

FIRECODE: A Computational Chemistry Workflow Driver and Modular Hub

Nicolò Tampellini ¹ ¶

¹ Massachusetts Institute of Technology, USA ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Computational chemistry, the branch of this science dedicated to the modeling of molecular systems for the extraction of properties of interest, is experiencing a recent renaissance thanks to impressive advances in theoretical models enabling ever-faster simulations of complex processes via density functional theory (DFT), semiempirical, and machine-learned interatomic potentials (MLIPs). However, the reduced computational cost will only translate into faster and more robust modeling with the help of automation frameworks, where trivial human intervention does not become the new bottleneck of computational pipelines. FIRECODE aims at filling this need by providing a modular framework for generating and processing molecular ensembles.

Statement of need

FIRECODE is a modular workflow driver for computational chemistry written in Python. The code interfaces with a series of fast modern calculators (xTB ([Bannwarth et al., 2019](#)), tblite ([GitHub - Tblite/Tblite; Katbashev et al., 2025](#)), AIMNET2 ([Anstine et al., 2024](#)), UMA ([Wood et al., 2025](#)), usually via ASE ([Ase / Ase · GitLab — Gitlab.com; Bahn & Jacobsen, 2002; Larsen et al., 2017](#))) and exposes a series of workflows and utilities for geometry optimization, frequency analysis, one- and two-ended transition state search, and conformational search. When possible, these routines are implemented in a calculator-independent way, and almost all of these can be arbitrarily chained to one another to define a complex workflow.

The software was born out of necessity over the last five years of research in asymmetric catalysis: FIRECODE was designed to streamline the necessary use of multiple software and routines in the computational modeling of small molecules, in a single centralized hub. The program can be called via command line with a minimal plain text input file defining the workflow, or used as a Python library in jupyter notebooks. The modular nature of the software enables rapid addition of new features, facilitating external contributions, while also retaining a small installation footprint for the core library. The concise input syntax also offers a streamlined definition of workflows and software orchestration with no coding experience required.

State of the field

While a manifold of computational chemistry software exists, workflow utilities are less common, particularly in the free and open source community. Recently, version 6.0 of ORCA ([Neese et al., 2020; Neese, 2022; Tetenberg et al., 2026](#)) (free for academic use) started offering compound job functionality, which enables the chained execution of some core ORCA modules. The code is however not open source and functionality is limited to what is included in the ORCA program (*i.e.* no MLIPs, no similarity pruning, no ensemble processing...). While

39 ORCA workflows are more extensively implemented in WEASEL ([Faccts](#)), this is commercial,
40 closed-source software.

41 The excellent AQME software ([Alegre-Requena et al., 2023](#)) features the most similar design
42 philosophy to the one of this work. Key differences with FIRECODE, besides code architecture,
43 lie in: 1) the main interaction platform of AQME happening via jupyter notebooks, while the
44 software of this work favors the use of plain text input files, keeping the inputs as minimal
45 as possible. 2) The present software having a reactivity-centered set of features, chiefly via
46 fast semiempirical and ML calculators, over AQME's focus on DFT and on the generation of
47 ensemble properties like spectra and molecular descriptors.

48 Other examples of related software, targeting the exposure of high-level primitives over
49 structured workflows, include Cuby ([GitHub - Berquist/Cuby](#); [Řezáč, 2016](#)) (MD and high-level
50 DFT) written in Ruby, and the popular ASE ([Ase / Ase · GitLab — Gitlab.com](#); [Bahn &
51 Jacobsen, 2002](#); [Larsen et al., 2017](#)), which is extensively used as a lower level interface for
52 interacting with calculators in FIRECODE.

53 The development of this software from scratch was motivated by multiple reasons. First, a
54 modern and flexible hub for running computational chemistry workflows, particularly tailored
55 to modeling chemical reactivity and MLIPs, was lacking. While the benefit of developing
56 standalone applications for specific tasks remains, their integration in a single hub permits
57 automation and lowers the barrier of entry for adoption of both individual tools and workflows
58 alike. Second, modern machine-learned interatomic potentials (MLIPs) are more conveniently
59 ran via Python libraries like torch ([GitHub - Pytorch/Pytorch](#)), benefitting greatly from a
60 Python-based workflow manager. Additional features like implicit delta solvation for gas phase-
61 trained MLIPs ¹ and numerical quasi-RRHO thermochemistry ([Grimme, 2012](#)) are additional
62 core library functionalities that were not available in other actively maintained modules at the
63 time of development.

64 Software design

65 The software is developed with the philosophy of maintaining the greatest amount of
66 functionality and module interoperability with the minimal list of dependencies, which should
67 ideally be as modern and lean as possible. All core dependencies are distributed via pypi,
68 keeping the installation of the minimal version of the program down to a single command (`uv
69 pip install firecode`).

70 Non-essential dependencies can be installed based on the desired calculator and interfaces to
71 be used, all via conda.

72 The software implements various calculators (xTB ([Bannwarth et al., 2019](#)), tblite ([GitHub -
73 Tblite/Tblite](#); [Katbashev et al., 2025](#)), AIMNET2 ([Anstine et al., 2024](#)), UMA ([Wood et al.,
74 2025](#))) via ASE ([Ase / Ase · GitLab — Gitlab.com](#); [Bahn & Jacobsen, 2002](#); [Larsen et al., 2017](#)),
75 which can be used interchangeably in various ensemble routines. Other core libraries leveraged
76 throughout the codebase are numpy ([GitHub - Numpy/Numpy](#)) for algebraic manipulations,
77 networkx ([GitHub - Networkx/Networkx](#); [Hagberg, Schult, et al., 2008](#); [Hagberg, Swart, et
78 al., 2008](#)) for graph utilities and prism_pruner ([GitHub - Ntampellini/Prism_pruner](#)) for the
79 removal of duplicates. Sella ([GitHub - Zadorlab/Sella](#); [Hermes et al., 2019, 2021, 2022](#))
80 provides a saddle point optimizer, and rdkit ([GitHub - Rdkit/Rdkit](#); [Greg Landrum, 2026](#)) is
81 used for ETKDG ([Riniker & Landrum, 2015](#)) conformational searches and SMARTS substructure
82 matching. The ML-FSM library ([GitHub - Thegomeslab/ML-FSM](#); [Marks & Gomes, 2025](#))
83 provides a two-ended TS search utility complementing ASE's implementation of the NEB method
84 ([Ásgeirsson et al., 2021](#)).

¹That is, a calculator providing the energy and gradient differences between an atomic structure in an implicit solvent and the same structure in vacuum.

85 Other standalone conformational search software is instead interfaced via subprocess calls
86 and file-based I/O, like CREST ([Pracht et al., 2020, 2024](#)), ORCA's GOAT ([Souza, 2025](#)) and
87 racerts ([GitHub - Digital-Chemistry-Laboratory/Racerts](#); [Schmid et al., 2026](#)).

88 The modular nature of the calculators and interfaces allows the definition of complex workflows
89 in plain text files (input.txt):

```
firecode input.txt -n test_run
```

90 A second standalone executable exposes the core optimizer routines acting directly on structure
91 files:

```
firecode_opt conformers.xyz --opt --freq --solvent toluene [...]
```

92 Contributions by way of adding new routines (or interfaces) to the codebase is made
93 straightforward by design: individual "operators" (routines) are simply functions reading a
94 molecular .xyz file and returning the filename of the processed ensemble. Run information
95 can be accessed from the Embedder class or via environment variables.

```
def center_operator(filename: str, embedder: Embedder) -> str:
    """Example operator centering the molecule."""

    # the Embedder class stores global information
    embedder.avail_gpus # 1
    embedder.options.solvent # "ch2cl2"
    embedder.options.T # 298.15

    # get a copy of the molecule of interest
    mol = embedder.mols[filename]

    # center coordinates
    mol.coords[0] -= np.mean(mol.coords[0], axis=0)

    # save any data you might need later
    embedder.options.centered = True
    embedder.options.last_operator = "center"

    # write to global log
    embedder.log(f"--> Center operator: centered molecule {mol.basename}")

    # write outfile and return its name
    outfile = f"{mol.basename}_centered.xyz"
    mol.to_xyz(outfile)

    return outfile
```

96 Research impact statement

97 Since its earliest versions in 2021 (formerly [TSCoDe](#)), this software proved essential in
98 orchestrating asymmetric catalysis workflows with ensembles up to thousands of structures
99 ([Huth et al., 2024](#); [Marvich et al., 2026](#); [Rozema et al., 2025](#); [Tampellini et al., 2024, 2025](#);
100 [Tampellini, Choi, et al., 2025](#)). Adoption is slowly starting to grow with pypi reporting
101 ~150 downloads per month at the present time. Part of the core code was exported to
102 the prism_pruner library ([GitHub - Ntampellini/Prism_pruner](#)) to facilitate its adoption by
103 the computational chemistry startup Rowan ([Rowan | ML-Powered Molecular Design and](#)
104 [Simulation — Rowansci.com](#)) (not affiliated with the author).

AI usage disclosure

Generative AI tools (Claude, Gemini) were used in some cases to generate first drafts of Python functions that were then manually curated line-by-line and interfaced with the rest of the codebase by hand. No generative AI tools were used in the writing of this manuscript.

Acknowledgements

We acknowledge years of research support from both Alma Mater Studiorum - University of Bologna (Prof. Giorgio Bencivenni and Prof. Paolo Righi) and Yale University (Prof. Scott J. Miller). These environments provided both the computational resources to run and develop the software as well as the primary problems it was designed to address and was stress-tested against.

References

- Alegre-Requena, J. V., Sowndarya S. V, S., Pérez-Soto, R., Alturaifi, T. M., & Paton, R. S. (2023). AQME: Automated quantum mechanical environments for researchers and educators. *WIREs Computational Molecular Science*, 13(5), e1663. <https://doi.org/10.1002/wcms.1663>
- Anstine, D., Zubatyuk, R., & Isayev, O. (2024). AIMNet2: A neural network potential to meet your neutral, charged, organic, and elemental-organic needs. This content is a preprint and has not been peer-reviewed. *ChemRxiv*. <https://doi.org/10.26434/chemrxiv-2023-296ch-v2>
- Ase / ase · GitLab — [gitlab.com](https://gitlab.com/ase/ase). <https://gitlab.com/ase/ase>.
- Ásgeirsson, V., Birgisson, B. O., Björnsson, R., Becker, U., Neese, F., Riplinger, C., & Jónsson, H. (2021). Nudged elastic band method for molecular reactions using energy-weighted springs combined with eigenvector following. *Journal of Chemical Theory and Computation*, 17(8), 4929–4945. <https://doi.org/10.1021/acs.jctc.1c00462>
- Bahn, S. R., & Jacobsen, K. W. (2002). An object-oriented scripting interface to a legacy electronic structure code [Article]. *Comput. Sci. Eng.*, 4(3), 56–66. <https://doi.org/10.1109/5992.998641>
- Bannwarth, C., Ehlert, S., & Grimme, S. (2019). GFN2-xTB—an accurate and broadly parametrized self-consistent tight-binding quantum chemical method with multipole electrostatics and density-dependent dispersion contributions. *Journal of Chemical Theory and Computation*, 15(3), 1652–1671. <https://doi.org/10.1021/acs.jctc.8b01176>
- Faccts. Weasel - FACCTs — [faccts.de](https://www.faccts.de/weasel/). <https://www.faccts.de/weasel/>.
- GitHub - berquist/cuby: A copy of cuby, a computational chemistry framework written in Ruby. — [github.com](https://github.com/berquist/cuby). <https://github.com/berquist/cuby>.
- GitHub - digital-chemistry-laboratory/racerts: A python package for efficient conformer sampling for transition states — [github.com](https://github.com/digital-chemistry-laboratory/racerts). <https://github.com/digital-chemistry-laboratory/racerts>.
- GitHub - networkx/networkx: Network Analysis in Python — [github.com](https://github.com/networkx/networkx). <https://github.com/networkx/networkx>.
- GitHub - ntampellini/prism_pruner: Similarity pruning engine for conformational ensembles — [github.com](https://github.com/ntampellini/prism_pruner). https://github.com/ntampellini/prism_pruner.
- GitHub - numpy/numpy: The fundamental package for scientific computing with Python. — [github.com](https://github.com/numpy/numpy). <https://github.com/numpy/numpy>.

- 148 GitHub - pytorch/pytorch: Tensors and Dynamic neural networks in Python with strong GPU
149 acceleration — github.com. <https://github.com/pytorch/pytorch>.
- 150 GitHub - rdkit/rdkit: The official sources for the RDKit library — github.com. <https://github.com/rdkit/rdkit>.
- 151
- 152 GitHub - tblite/tblite: Light-weight tight-binding framework — github.com. <https://github.com/tblite/tblite>.
- 153
- 154 GitHub - thegomeslab/ML-FSM: Public FSM implementation with support for ML-based
155 potential energy surfaces — github.com. <https://github.com/thegomeslab/ML-FSM>.
- 156 GitHub - zadorlab/sella: A Python software package for saddle point optimization and
157 minimization of atomic systems. — github.com. <https://github.com/zadorlab/sella>.
- 158 Greg Landrum, B. K., Paolo Tosco. (2026). Rdkit/rdkit: 2026_03_3 (Q1 2026). <https://doi.org/10.5281/zenodo.20446949>
- 159
- 160 Grimme, S. (2012). Supramolecular binding thermodynamics by dispersion-corrected density
161 functional theory. *Chemistry - A European Journal*, 18(32), 9955–9964. <https://doi.org/10.1002/chem.201200497>
- 162
- 163 Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics,
164 and function using NetworkX. *Proceedings of the 7th Python in Science Conference*, 11–15.
165 <https://doi.org/10.25080/TCWV9851>
- 166 Hagberg, A. A., Swart, P. J., & Schult, D. A. (2008). *Exploring network structure, dynamics,
167 and function using NetworkX*. Los Alamos National Laboratory (LANL). <https://www.osti.gov/biblio/960616>
- 168
- 169 Hermes, E. D., Sargsyan, K., Najm, H. N., & Zádor, J. (2019). Accelerated saddle point
170 refinement through full exploitation of partial hessian diagonalization. *Journal of Chemical
171 Theory and Computation*, 15(11), 6536–6549. <https://doi.org/10.1021/acs.jctc.9b00869>
- 172 Hermes, E. D., Sargsyan, K., Najm, H. N., & Zádor, J. (2021). Geometry optimization speedup
173 through a geodesic approach to internal coordinates. *The Journal of Chemical Physics*,
174 155(9). <https://doi.org/10.1063/5.0060146>
- 175 Hermes, E. D., Sargsyan, K., Najm, H. N., & Zádor, J. (2022). Sella, an open-source
176 automation-friendly molecular saddle point optimizer. *Journal of Chemical Theory and
177 Computation*, 18(11), 6974–6988. <https://doi.org/10.1021/acs.jctc.2c00395>
- 178 Huth, S. E., Tampellini, N., Guerrero, M. D., & Miller, S. J. (2024). Catalytic enantioselective
179 sulfoxidation of functionalized thioethers mediated by aspartic acid-containing peptides.
180 *Organic Letters*, 26(32), 6872–6877. <https://doi.org/10.1021/acs.orglett.4c02452>
- 181 Katbashev, A., Stahn, M., Rose, T., Alizadeh, V., Friede, M., Plett, C., Steinbach, P., &
182 Ehlert, S. (2025). Overview on building blocks and applications of efficient and robust
183 extended tight binding. *The Journal of Physical Chemistry A*, 129(10), 2667–2682.
184 <https://doi.org/10.1021/acs.jpca.4c08263>
- 185 Larsen, A. H., Mortensen, J. J., Blomqvist, J., Castelli, I. E., Christensen, R., Duřak, M.,
186 Friis, J., Groves, M. N., Hammer, B., Hargus, C., Hermes, E. D., Jennings, P. C.,
187 Jensen, P. B., Kermode, J., Kitchin, J. R., Kolsbjerg, E. L., Kubal, J., Kaasbjerg, K.,
188 Lysgaard, S., ... Jacobsen, K. W. (2017). The atomic simulation environment—a python
189 library for working with atoms. *Journal of Physics: Condensed Matter*, 29(27), 273002.
190 <http://stacks.iop.org/0953-8984/29/i=27/a=273002>
- 191 Marks, J., & Gomes, J. (2025). Incorporation of internal coordinates interpolation into the
192 freezing string method. *Journal of Chemical Theory and Computation*, 21(23), 12110–12120.
193 <https://doi.org/10.1021/acs.jctc.5c01492>
- 194 Marvich, H. M., Langner, O. C., Tampellini, N., & Miller, S. J. (2026). A bifunctional aminoxyl-

- bipyridine peptide catalyst for the atroposelective copper-catalyzed aerobic oxidation of biaryl diols. *Journal of the American Chemical Society*, 148(22), 23032–23041. <https://doi.org/10.1021/jacs.6c04560>
- Neese, F. (2022). Software update: The ORCA program system—version 5.0. *WIREs Computational Molecular Science*, 12(5), e1606. <https://doi.org/10.1002/wcms.1606>
- Neese, F., Wennmohs, F., Becker, U., & Riplinger, C. (2020). The ORCA quantum chemistry program package. *The Journal of Chemical Physics*, 152(22), 224108. <https://doi.org/10.1063/5.0004608>
- Pracht, P., Bohle, F., & Grimme, S. (2020). Automated exploration of the low-energy chemical space with fast quantum chemical methods. *Physical Chemistry Chemical Physics*, 22(14), 7169–7192. <https://doi.org/10.1039/C9CP06869D>
- Pracht, P., Grimme, S., Bannwarth, C., Bohle, F., Ehlert, S., Feldmann, G., Gorges, J., Müller, M., Neudecker, T., Plett, C., Spicher, S., Steinbach, P., Wesołowski, P. A., & Zeller, F. (2024). CREST—a program for the exploration of low-energy molecular chemical space. *The Journal of Chemical Physics*, 160(11), 114110. <https://doi.org/10.1063/5.0197592>
- Řezáč, J. (2016). Cuby: An integrative framework for computational chemistry. *Journal of Computational Chemistry*, 37(13), 1230–1237. <https://doi.org/10.1002/jcc.24312>
- Riniker, S., & Landrum, G. A. (2015). Better informed distance geometry: Using what we know to improve conformation generation. *Journal of Chemical Information and Modeling*, 55(12), 2562–2574. <https://doi.org/10.1021/acs.jcim.5b00654>
- Rowan | ML-Powered Molecular Design and Simulation — rowansci.com. <https://rowansci.com/>.
- Rozema, S. D., Tampellini, N., Rein, J., Sigman, M. S., Lin, S., & Miller, S. J. (2025). Experimental lineage and computational analysis of a general aminoxyl-based oxidation catalyst: Generality from substrate-specific interactions. *ACS Catalysis*, 15(20), 17548–17557. <https://doi.org/10.1021/acscatal.5c05893>
- Schmid, S. P., Seng, H., Kläy, T., & Jorner, K. (2026). Rapid generation of transition-state conformer ensembles via constrained distance geometry. *Journal of Chemical Information and Modeling*, 66(5), 2777–2790. <https://doi.org/10.1021/acs.jcim.5c02794>
- Souza, B. de. (2025). GOAT: A global optimization algorithm for molecules and atomic clusters. *Angewandte Chemie International Edition*, 64(18), e202500393. <https://doi.org/10.1002/anie.202500393>
- Tampellini, N., Choi, E. S., & Miller, S. J. (2025). Peptide-catalyzed asymmetric amination of sulfenamides enabled by DFT-guided catalyst optimization. *Journal of the American Chemical Society*, 147(44), 41122–41129. <https://doi.org/10.1021/jacs.5c15618>
- Tampellini, N., Mercado, B. Q., & Miller, S. J. (2024). Scaffold-oriented asymmetric catalysis: Conformational modulation of transition state multivalency during a catalyst-controlled assembly of a pharmaceutically relevant atropisomer. *Chemistry – A European Journal*, 30(30), e202401109. <https://doi.org/10.1002/chem.202401109>
- Tampellini, N., Mercado, B. Q., & Miller, S. J. (2025). Enantiocontrolled cyclization to form chiral 7- and 8-membered rings unified by the same catalyst operating with different mechanisms. *Journal of the American Chemical Society*, 147(5), 4624–4630. <https://doi.org/10.1021/jacs.4c17080>
- Tetenberg, T., Neugebauer, H., Plett, C., Santhosh, N., Bursch, M., & Riplinger, C. (2026). ORCA meets python - the ORCA python interface OPI. *Journal of Chemical Theory and Computation*, 22(10), 4951–4967. <https://doi.org/10.1021/acs.jctc.5c02141>
- Wood, B. M., Dzamba, M., Fu, X., Gao, M., Shuaibi, M., Barroso-Luque, L., Abdelmaqsoud,

242 K., Gharakhanyan, V., Kitchin, J. R., & Levine, D. S. (2025). UMA: A family of universal
243 models for atoms. *arXiv Preprint*. <https://doi.org/10.48550/arXiv.2506.23971>

DRAFT